

Bar-Ilan University

אוניברסיטת בר-אילן

M.Sc. Research Proposal

---

# Rule Learning with Small Language Models
## in Interactive Puzzle Games

## למידת חוקי משחק באמצעות מודלי שפה קטנים
## במשחקי חידה אינטראקטיביים

---

*Student:*
Adam Celermajer
אדם סלרמאג׳ר

*Supervisors:*
Prof. Gal Kaminka
Prof. Yoav Goldberg

Department of Computer Science and Artificial Intelligence
המחלקה למדעי המחשב ובינה מלאכותית
Bar-Ilan University, Ramat Gan, Israel

October 2026

# Contents


**1 Introduction** **1**

**2 The Task: Learning Rules from Multimodal Interaction** **1**

2.1 PuzzleScript as an experimental environment . . . . . 1

2.2 A concrete example: Modality . . . . . 2

2.3 The capabilities the agent must acquire . . . . . 2

**3 Related Work and Research Position** **3**

**4 Research Objective and Hypotheses** **3**

**5 Completed Research Foundation and Initial Results** **4**

5.1 An executable environment and diagnostic evaluation platform . . . . . 4

5.2 Explicit rule induction and verification . . . . . 5

5.3 A model-and-human goal-recognition study . . . . . 5

5.4 Persistent direct play and isolated solver evolution . . . . . 5

5.5 Demonstrated whole-game solving and mechanics recovery . . . . . 6

5.6 A specific, measurable small-model gap . . . . . 7

**6 Experimental Program: From Rule Acquisition to Verified Solutions** **7**

6.1 First: compare the Qwen 3.8 family on the same task . . . . . 7

6.2 Second: locate the bottleneck through controlled interventions . . . . . 8

6.3 Third: build and ablate the support that makes the small model solve . . . . . 8

6.4 Validation, efficiency and generalization . . . . . 9

**7 Work Plan and Deliverables** **9**

**References** **10**

# 1 Introduction

**This thesis will enable a small language model to learn game rules from images, grid observations and interaction, and use those rules to solve interactive puzzles.** Our primary target is Qwen 3.8-27B. The deliverable is a working agent that discovers the controls, goal and mechanics of a new game, constructs a useful world model, and produces solutions that succeed when executed independently. We will use controlled experiments to identify the capabilities the harness must support and demonstrate the resulting improvement on games held out from development.

The task is more demanding than solving a puzzle with stated rules. In our main game-playing conditions, the agent has access to both rendered PNG images and integer grid observations, together with anonymous action identifiers. Images provide visual evidence about objects, terrain and targets; they do not state the transition rules, action meanings or win condition. The agent must combine visual and symbolic evidence, design informative experiments, infer what counts as success, and plan with incomplete knowledge. Later levels test whether the learned rules survive changes in layout and combinations of mechanics. This is the skill-acquisition setting motivating ARC-AGI-3 (ARC Prize Foundation, 2026; Chollet, 2019).

We have already established the research foundation for this objective: an ARC-compatible PuzzleScript environment, diagnostic evaluation artifacts, an explicit rule-learning agent, a goal-recognition dataset with model and human responses, persistent direct-play baselines, and an isolated adaptation of Agentic Variation Operators (AVO) (Chen et al., 2026). A web workbench joins experiment configuration, execution, replay and artifact inspection. These components make it possible to trace a solver's result back to the observations, experiments and revisions that produced it.

The existing results establish both feasibility and a concrete research target. AVO-generated solvers have completed all levels of Modality, Knightoban and IceCrates in fresh environments, including solvers produced by DeepSeek V4.1 Flash. In contrast, the completed Qwen 3.8-27B direct-play pilot achieved 0/4, 0/6 and 3/6 levels on those games, respectively. Its recorded behavior reveals specific failures in action interpretation, goal recognition and hypothesis revision. We will close this gap by turning those diagnoses into tested components of a small-model learning system.

# 2 The Task: Learning Rules from Multimodal Interaction

## 2.1 PuzzleScript as an experimental environment

PuzzleScript is a language for turn-based grid games (Lavelle, 2013). A game defines objects and their appearance, rewrite rules for state transitions, a win condition, and an ordered sequence of levels. The resulting tasks include box-pushing puzzles, sliding motion, objects with different movement rules, mode changes, and interactions whose effects depend on terrain or history. Their compact appearance makes the mechanics inspectable by an evaluator while leaving substantial discovery and planning work for an agent.

Our collection contains 206 game entries. Experiments use curated, versioned playable subsets with explicit evaluation coverage. The agent sees neither the game identity nor the source. For the local environment, `visible-cell-v1` assigns an anonymous integer to each rendered tile texture. The direct player receives the rendered PNG alongside the grid at each turn; AVO makes rendered observations available through its research interface. A model can therefore recognize visual patterns and use them to propose rules, rather than reason only over anonymous numbers. Object-name legends and action meanings are withheld. Identical visible textures may conceal different underlying states, so the agent may need observation history rather than

a transition model based on the latest frame alone.

## 2.2 A concrete example: Modality

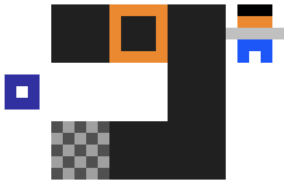
Figure 1 illustrates the distinction between seeing a board and understanding a game. In Modality, the player cannot move directly between white and black terrain; a checkered tile connects them. Crate pushing depends on compatible terrain, the goal is to place crates on targets, and `ACTION7` undoes a move. The agent must infer these facts from their effects. Calling undo a jump changes its interpretation of earlier transitions; calling a crate the goal sends its plans toward the wrong objective.



(a) First-level image

| | | | | |
|---|---|---|---|---|
| 0 | 1 | 2 | 2 | 2 |
| 3 | 1 | 4 | 2 | 5 |
| 1 | 1 | 1 | 2 | 2 |

(b) Anonymous tile grid



(c) A later layout

**Figure 1:** Modality observations. For the evaluator, the initial IDs denote a target (0), white floor (1), black floor (2), crate (3), bridge (4) and player on black floor (5). These meanings are not supplied to the agent. Later levels require reusing the inferred mechanics on new configurations.

## 2.3 The capabilities the agent must acquire

We decompose the task into the seven capabilities in Table 1. The decomposition connects the experimental measurements to concrete design decisions. It also distinguishes a failure to learn a game from a failure of the software protocol.

| Capability | Required behavior | Diagnostic evidence |
|---|---|---|
| Perception | Ground objects in images and grids; track changing textures and hidden state. | Object tracking and next-frame predictions. |
| Action interpretation | Discover movements, undo, selection and commitment. | Controlled action tests and the agent's stated control model. |
| Rule induction | Learn conditional effects, including interactions. | Predicted versus observed transitions; tests distinguishing rival rules. |
| Goal recognition | Infer the actual win condition. | Goal statements and hidden, elicited and supplied-goal conditions. |
| Planning | Find legal solutions and recover from dead ends. | Fresh solver evaluations, search traces and recovery actions. |
| Transfer | Reuse and refine knowledge across levels. | Performance when familiar mechanics appear in new layouts. |
| Credit assignment | Explain why an action sequence produced a win. | Post-win reconstruction checked against the winning transition. |

**Table 1:** Capabilities supporting rule learning and its use in planning.

These capabilities interact. A wrong goal can invalidate a correct movement model; an incorrect interpretation of undo can make a recoverable planning error appear permanent. The

proposed agent will maintain explicit links between observations, hypotheses and plans so that contradictory evidence causes a revision rather than a new local explanation for every failure.

## 3 Related Work and Research Position

**Interactive skill acquisition.** ARC-AGI-3 evaluates learning through interaction with unfamiliar environments, including exploration, goal inference and planning (ARC Prize Foundation, 2026). PuzzleJAX establishes PuzzleScript as a source of diverse reasoning and learning tasks (Earle et al., 2025). Our platform retains the original engine as the transition authority and exposes an ARC-compatible interface. This lets us study how an agent acquires mechanics while separately measuring predictive knowledge and executable solutions. The experiments proposed here concern PuzzleScript tasks; they do not claim a score on the official ARC-AGI-3 benchmark.

**Agent interfaces and persistent learning.** ReAct interleaves reasoning and actions to support grounded decision making (Yao et al., 2023). SWE-agent demonstrates that the interface provided to a language model affects its ability to carry out a task (Yang et al., 2024). We investigate this principle under a different constraint: the model must learn the task's mechanics and objective rather than follow a stated specification. Our direct-play and coding-agent runners provide an existing basis for controlled comparisons of memory, tools, observation formats and experiment structure.

**Executable models and evolutionary improvement.** WorldCoder learns a program representing environmental dynamics and uses interaction to refine it (Tang et al., 2024). AVO uses autonomous coding agents as variation operators that propose and improve implementations with execution feedback (Chen et al., 2026). Our adaptation combines game research, strategy evolution and a separate fresh-solver evaluation. This creates a testable connection between a model's hypotheses and an executable artifact: a candidate earns its score only when its solver succeeds independently.

**Prior knowledge and the small-model question.** Human game learning depends partly on object and game conventions (Dubey et al., 2018). Our traces suggest that recognizing conventions such as undo and box pushing can simplify discovery substantially. However, behavioral familiarity alone cannot establish what appeared in a model's training data. We will compare models within the Qwen 3.8 family and probe their initial interpretations, then test whether explicit exploration and verification allow the 27B model to acquire the missing knowledge during play.

## 4 Research Objective and Hypotheses

**We will build a small-model rule-learning agent that infers game dynamics from image and grid transitions and produces working puzzle solutions.** Qwen 3.8-27B is the primary development model. We will keep its weights fixed while developing the harness, so improvements can be attributed to the agent system. Larger models provide diagnostic comparisons and successful learning traces; the final small-model condition will not rely on a larger model selecting its actions or revising its solver at evaluation time.

The harness will support a complete learning cycle: formulate a hypothesis, choose a discriminating experiment, compare a predicted transition with the real outcome, revise the world model, and plan using the verified knowledge. Memory will retain the evidence for a rule and

its unresolved cases, not only a conclusion. In the coding condition, learned knowledge will be converted into a solver that is tested from a fresh reset.

Three research questions guide construction:

1. Which capabilities distinguish correct rule acquisition and revision from the observed failures of the small model?
2. Which of these capabilities can be supplied or strengthened through exploration, memory, verification and planning support?
3. Do the learned rules predict held-out transitions and support reliable puzzle solutions, including on games held out from harness development?

**Testable hypotheses.**

**H1: Learning structure.** An explicit experiment-and-revision loop improves rule prediction and whole-game success over the direct-play baseline with the same small model and provider.

**H2: Goal and control bottlenecks.** Correcting goal recognition or action interpretation removes a substantial part of the observed failures. Separate interventions will measure which component matters in each game.

**H3: Externalized planning.** Executable world models, deterministic search and fresh evaluation improve solution reliability beyond verbal planning alone.

**H4: Model-family differences.** Under matched conditions, Qwen 3.8 Flash, 27B and Max differ in their ability to discover and revise the decisive rules. Their traces will identify concrete capabilities for the small-model harness to support.

# 5 Completed Research Foundation and Initial Results

The project has developed an integrated platform for observing, measuring and improving rule-learning agents. The completed work is organized below by research contribution and evidence, rather than by elapsed calendar time.

## 5.1 An executable environment and diagnostic evaluation platform

The implementation separates the game engine from the agent interface. A Node.js runtime executes the PuzzleScript rules; a Python service adapts it to the ARC interaction contract, including resets, actions and scorecards. Shared adapters let the same agent code address the local service or an official ARC backend. This separation permits changes to agents and evaluation without replacing the game dynamics.

We also built evaluator artifacts that measure knowledge independently of winning. The current benchmark set covers 21 games with replay-verified reference trajectories and ten unique next-observation probes per game. Probe construction covers distinct rule families and extends exploration when a short solution provides insufficient transitions. The scoring records exact-frame accuracy, cell accuracy and transition-edit measures. A separate closeness measure reports progress on a verified solution trace and explicitly marks off-trace states as unavailable. These tools allow us to ask whether an agent has learned useful dynamics even when it has not yet found a solution.

**Experiment infrastructure.** The workbench launches bounded batches, records their settings and process ownership, and retains frames, transitions, note histories, model requests, solver files and scores. Its replay interface connects an action to the available agent reasoning and artifacts. Research actions and fresh solver evaluations are inspected separately. Persistent

manifests support continuation and recovery while preserving the original attempt. Together, these components provide a practical experimental system for repeated, auditable comparisons.

### 5.2 Explicit rule induction and verification

We implemented a rule-learning agent that records state–action–state transitions, asks a language model to propose rules, compiles them into an executable logical representation, and checks them against accumulated evidence. A planner searches with verified rules; model-generated subgoals support exploration when no rule-based plan is available.

Saved Sokoban experiments contain verified movement, collision and crate-push rules. The important methodological result is an implemented link between a verbal hypothesis, an executable prediction and a recorded transition. This provides a baseline for the proposed learning cycle. The later coding-agent approach extends the representation to mechanics whose state, interactions or search procedures are easier to express directly in programs. We will compare these representations through their predictions and solutions.

### 5.3 A model-and-human goal-recognition study

We designed and completed data collection for a separate study of goal recognition across 50 curated PuzzleScript games. Four models—Claude Opus 4.8, GPT-5.5, Kimi K2.6 and DeepSeek V4 Pro—were asked to describe the goal and win condition. The collection varies both the available evidence (an initial frame or an initial frame followed by three random actions) and modality (text or text plus image; DeepSeek is text-only). The resulting analysis set contains 700 model predictions. A human study collected 114 goal descriptions from 38 participants across 48 games.

| Evidence condition | Text models | Text + image models | Predictions |
|---|---|---|---|
| Initial frame | 4 | 3 | 350 |
| Initial frame + three actions | 4 | 3 | 350 |
| Human responses | 38 participants; 48 games | | 114 |

**Table 2:** Completed goal-recognition data collection. Correctness annotation and the model–human accuracy comparison form the analysis phase.

The study includes evidence preparation, reusable prompt builders, model-call records, a response viewer and a review tool supporting correct, partial and incorrect judgments. Data collection is complete; correctness annotation is the next analysis step, so no accuracy claim is made here.

The collected responses already expose a useful design issue. Initial-frame text prompts used game-specific glyphs, whereas trajectory prompts used anonymous IDs. A Sokoban-style glyph can activate the right genre prior but also the wrong interpretation of an object. For example, a model interpreted `@` as a player when it denoted a crate on a target. We will analyze the existing data with this representation difference explicit and use matched representations in subsequent evidence comparisons. This finding connects the standalone study to the proposed goal-recognition interventions.

### 5.4 Persistent direct play and isolated solver evolution

**Direct play.** The direct player maintains one model session across the game. The baseline+ condition adds persistent note tools and a strategy file while retaining the same action and

observation loop. This gives a reproducible reference for learning through ordinary interaction, with recorded transitions and immutable note revisions.

**AVO adaptation.** We implemented a distinct research–optimization–evaluation architecture. An inner coding agent explores the game, compares continuations from saved checkpoints and constructs a solver. An outer optimizer improves strategies through a recorded candidate lineage. A trusted evaluator then executes each frozen solver from a fresh reset, without language-model calls. The candidate's fitness is the number of levels completed in this independent evaluation:

$$\text{strategy}_i \longrightarrow \text{research}_i \longrightarrow \texttt{solver}_i \longrightarrow \text{fresh evaluation.}$$

Model commands run in a Linux sandbox without access to game sources, private evaluation artifacts or network tools. The trusted controller owns environment and provider communication. The implementation also retains candidate strategies, inherited knowledge, evaluation feedback and final world-model reports. A report is evidence about stated understanding; its presence does not affect solver fitness.

### 5.5 Demonstrated whole-game solving and mechanics recovery

Table 3 reports representative best fresh evaluations. These are independently executed solvers, not scores inferred from research notes or from the model's claim that it has understood a game.

| Game | Player / optimizer | Levels | Solver actions |
|---|---|---|---|
| Modality | GPT-5.6 Luna / GPT-6 Astra | 4/4 | 263 |
| Knightoban | GPT-5.6 Luna / GPT-6 Astra | 6/6 | 255 |
| IceCrates | GPT-6 Astra / GPT-6 Astra | 6/6 | 138 |
| Modality | DeepSeek V4.1 Flash / same | 4/4 | 263 |
| Knightoban | DeepSeek V4.1 Flash / same | 6/6 | 216 |
| IceCrates | DeepSeek V4.1 Flash / same | 6/6 | 90 |

**Table 3:** Observed AVO whole-game successes. Rows report the best completed fresh evaluation from the corresponding campaign, not averages across repeated attempts. Model-role settings are retained in the run manifests.

We audited the discovered mechanics against the source rules after the runs. In Modality, controlled tests established terrain-dependent movement and pushing. In Knightoban, the agent separated move preview, spatial selection and action-key commitment and constructed a planner for interacting pieces. In IceCrates, experiments distinguished moving crates from walls, established water filling and the need to stop on the goal, and corrected a missing rule for pushing a two-crate train. After that correction, the independent solver completed all six levels.

This analysis demonstrates more than a final score: recorded experiments produced revisions that made an executable model useful for planning. The audit relies on contemporaneous strategies, transitions and solver code; reconstructed post-run reports are identified separately. It establishes the mechanics needed by the successful solvers, rather than claiming that every unobserved combination was exhaustively characterized.

The broader experiments also identify where generalization remains difficult. GPT-5.6 Luna solved Feeling Like Filling (6/6), Mowing (4/4) and Fatigued Square Maze (8/8) in both recorded repeats of the six-game comparison. Enqueue reached 8/14 levels with both the GPT-6 Luna and DeepSeek Flash configurations. These results supply successes and partial-progress cases for evaluating whether the proposed small-model improvements transfer across different mechanics.

### 5.6 A specific, measurable small-model gap

The completed Qwen 3.8-27B pilot used a pinned Alibaba provider, high effort, and baseline+ through Codex. The resulting outcomes are summarized in Table 4. Each run reached its action budget; the partial IceCrates result is therefore directly distinguishable from a startup or transport error.

| Game | Actions | Levels | Evidence for the next intervention |
|---|---|---|---|
| Modality | 100 | 0/4 | Undo misread; crate deadlock not recovered |
| Knightoban | 120 | 0/6 | Goal and special movement not identified |
| IceCrates | 120 | 3/6 | Undo identified; later mechanics unresolved |

**Table 4:** Completed 27B direct-play pilot. AVO and direct-play results are distinct conditions; their difference motivates, rather than replaces, matched comparisons.

The traces make the target precise. Earlier Modality attempts cleared the first level while interpreting the crate as a destination and undo as a backward jump. The accidental win reinforced an incorrect explanation. The pinned run did recognize a pushable box and a target, but retained the jump interpretation and treated a recoverable crate deadlock as permanent. In IceCrates, the same model identified undo and solved three levels in its first 27 actions. Thus, the research target is the reliable acquisition and revision of specific mechanics, not an assumption that the model has no game-playing ability.

We have also isolated interface effects. In a small controlled diagnostic, an enforced JSON action schema yielded zero note-tool calls in two checks; the corresponding checks without that schema each called the tool. A separate end-to-end check verified note writing and reading without the schema. The action parser now accepts valid actions surrounded by prose and requests a correction for unusable replies. Provider pinning and recorded request metadata reduce routing ambiguity; disclosed serving precision is recorded when available. These checks establish a better-controlled baseline for capability experiments. They are reported separately from game-learning outcomes.

## 6 Experimental Program: From Rule Acquisition to Verified Solutions

### 6.1 First: compare the Qwen 3.8 family on the same task

We will compare Qwen 3.8 Flash, 27B and Max on Modality with the same direct-player condition: Codex baseline+, Alibaba, high effort, image and grid observations, and a 100-action budget. Flash and Max runs were initiated on 13 October 2026. For any model that does not solve the game, a fresh AVO campaign will test whether executable modeling and strategy evolution unlock its capability. The same model will occupy the player and optimizer roles. Direct-play and AVO outcomes will remain separate, with their respective action, candidate, time and token budgets reported explicitly.

This experiment identifies the next useful intervention. If Max discovers the decisive rule but 27B does not, we will compare the experiments and revisions that led to it. If the same smaller model succeeds under AVO, we will isolate which support made the difference. If the family stalls on the same mechanic, we will target that shared bottleneck. Family differences do not isolate parameter count alone: training, architecture and serving conditions will be treated as additional possible explanations.

The pilot will then be expanded to Modality, Knightoban and IceCrates with independent repeats. These games supply distinct challenges in terrain, selection and commitment, and multi-

object sliding interactions. Existing successful solver records provide diagnostic comparisons, while the actual held-out evaluation will use fresh runs.

### 6.2 Second: locate the bottleneck through controlled interventions

We will use the capability decomposition to select interventions, rather than add several forms of assistance at once. Initial conditions will test:

1. **Goal recognition:** the goal is hidden, explicitly elicited and revised, or supplied by the evaluator.
2. **Action interpretation:** controls remain anonymous, or their true meanings are supplied while the rules and goal remain hidden.
3. **Mechanics and planning:** selected true mechanics are supplied diagnostically, or the model's learned mechanics are paired with a deterministic planner.

Supplying the truth is a diagnostic intervention, not the final task setting. A gain under a supplied-goal condition estimates how much incorrect goals contribute to failure; a residual gap motivates further tests of dynamics and planning. The final agent must acquire the hidden information itself.

For each failed attempt we will identify the first consequential mismatch between the model's belief and the observed transition, along with secondary causes such as failure to recover. Annotators will cite the supporting step, action and note rather than infer the cause from terminal reward alone. Independent human annotation of a sample will assess the reliability of the taxonomy; we will report agreement and disagreements rather than treat an automatic label as ground truth.

The existing goal-recognition corpus will be annotated in parallel using game-specific win conditions and a correct/partial/incorrect rubric. Model–human comparisons will describe the evidence and representation available in each condition. This analysis supplies an independent view of goal inference and helps choose appropriate scaffolds for interactive play.

### 6.3 Third: build and ablate the support that makes the small model solve

We will develop the harness from the diagnosed failures. Candidate components are a ledger linking hypotheses to evidence, controlled experiment selection, explicit transition predictions, revision when evidence contradicts a rule, and reconstruction of what caused each win. The ledger will distinguish observed, predicted and untested cases, preserving uncertainty instead of recording unknown moves as walls or no-ops.

The coding condition will support executable world models and search. The small model will infer and implement the mechanics; deterministic procedures will handle repeated simulation and path search. A mismatch between simulation and a real transition will invalidate the relevant rule or plan and trigger new research. Fresh solver evaluation will verify that the resulting program works without continued language-model guidance.

Each component will first be compared with the established baseline under matched conditions. We will then combine components that produce repeatable gains and remove them individually to determine their contribution. The comparison will record the actual observation history, context policy and tool inventory, since changing these silently can change the task. Interface transport comparisons are available when needed, but follow the family and bottleneck experiments rather than determine the initial direction.

### 6.4 Validation, efficiency and generalization

Let $L(m, h, g)$ be the number of levels completed by model $m$ with harness $h$ on game $g$, and let $n_g$ be the number of playable levels. We will report rule-prediction accuracy, whole-game success, $L/n_g$, actions, candidate evaluations, elapsed search time, tokens and measured API cost. For AVO, the solver outcome comes from the fresh frozen solver; research progress is an additional diagnostic. Next-observation accuracy and goal correctness measure learned knowledge separately from task completion.

Development uses the three diagnostic games. Before final evaluation, we will select and freeze a held-out subset from the curated collection, covering different mechanics and level structures. Game sources, verified solutions and source-derived annotations remain evaluator-only. Runs are independent, and no notes or solutions from a held-out game are inherited by its evaluated agent. We will report repeated-attempt outcomes and uncertainty, not only the best candidate.

The primary success criterion is that the fixed small model learns rule models that predict held-out transitions and enable complete solutions on games its baseline fails, with repeatable gains under declared resource budgets. A solver that improves only on a development layout is an intermediate result; the final contribution must demonstrate that the learning procedure transfers.

## 7 Work Plan and Deliverables

Table 5 defines the remaining work by verifiable deliverables. Family-comparison results will guide the interventions, with repeated experiments concentrated on promising conditions.

| Milestone | Work | Completion evidence |
|---|---|---|
| Family comparison | Direct play and AVO with Flash, 27B and Max; expand informative conditions with repeats. | Matched settings, outcomes, costs and decisive-rule trace comparisons. |
| Bottleneck analysis | Goal, control and mechanics interventions; annotate failure traces and the goal corpus. | Scored interventions, annotated evidence and agreement analysis. |
| Small-model solver | Implement and combine experiment, memory, verification and search support. | Whole-game successes by the fixed small model; component ablations. |
| Generalization | Freeze the harness and held-out games; run repeated independent evaluations. | Fresh solver scores, uncertainty and efficiency comparisons. |
| Thesis and release | Document the method, artifacts and limits. | Reproducible implementation, experiment records and thesis. |

**Table 5:** Milestones for the remaining research, organized around outputs.

**Resource management.** We will enforce action, candidate and wall-time budgets, record tokens and measured costs, and retain unsuccessful attempts. Infrastructure errors will be distinguished from capability outcomes. Direct-play and AVO comparisons will report both rule-learning gains and the resources used to obtain them.

**Expected contributions.** We will deliver a working rule-learning agent built around a small model, validated on held-out games. The research will also provide scored goal-recognition data, component ablations, and reproducible rule-prediction and fresh-solver evaluations.

## References


ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026. URL https://arxiv.org/abs/2603.24621.

Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026. URL https://arxiv.org/abs/2603.24517.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

Rachit Dubey, Pulkit Agrawal, Deepak Pathak, Thomas L. Griffiths, and Alexei A. Efros. Investigating human priors for playing video games. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. arXiv:1802.10217.

Sam Earle, Graham Todd, Yuchen Li, Ahmed Khalifa, Muhammad Umair Nasir, Zehua Jiang, Andrzej Banburski-Fahey, and Julian Togelius. PuzzleJAX: A benchmark for reasoning and learning. *arXiv preprint arXiv:2508.16821*, 2025. URL https://arxiv.org/abs/2508.16821.

Stephen Lavelle. PuzzleScript: An open-source HTML5 puzzle game engine. https://www.puzzlescript.net/, 2013. Software and domain-specific language.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.12275.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.